

Engineering Guide & Sprint Plan

PRODUCT NAME	Staye
DOCUMENT TITLE	Backend Infrastructure: Supabase to AWS Migration Runbook
TARGET ARCHITECTURE	Phase 1 (EC2, RDS PostgreSQL, Cognito, S3)
TARGET AUDIENCE	Backend Engineering & DevOps Team
PREPARED BY	Zenxvio Engineering Team
DATE	July 2026

Purpose of This Document

This runbook provides exhaustive, step-by-step instructions for the engineering team to migrate the Staye platform's backend infrastructure off Supabase and onto a production-grade AWS environment.

The migration is structured into **four distinct Sprints**. By the end of Sprint 4, the application will run as a Dockerized Next.js monolith on a free-tier EC2 instance, utilizing AWS RDS for data, Cognito for authentication, and S3 for storage.

Table of Contents

- 1. Migration Architecture Overview
- 2. Sprint 1: Cloud Foundation & Infrastructure Provisioning
- 3. Sprint 2: Codebase Refactoring (Moving off Supabase)
- 4. Sprint 3: Dockerization & CI/CD Pipeline
- 5. Sprint 4: Data Cutover & Go-Live

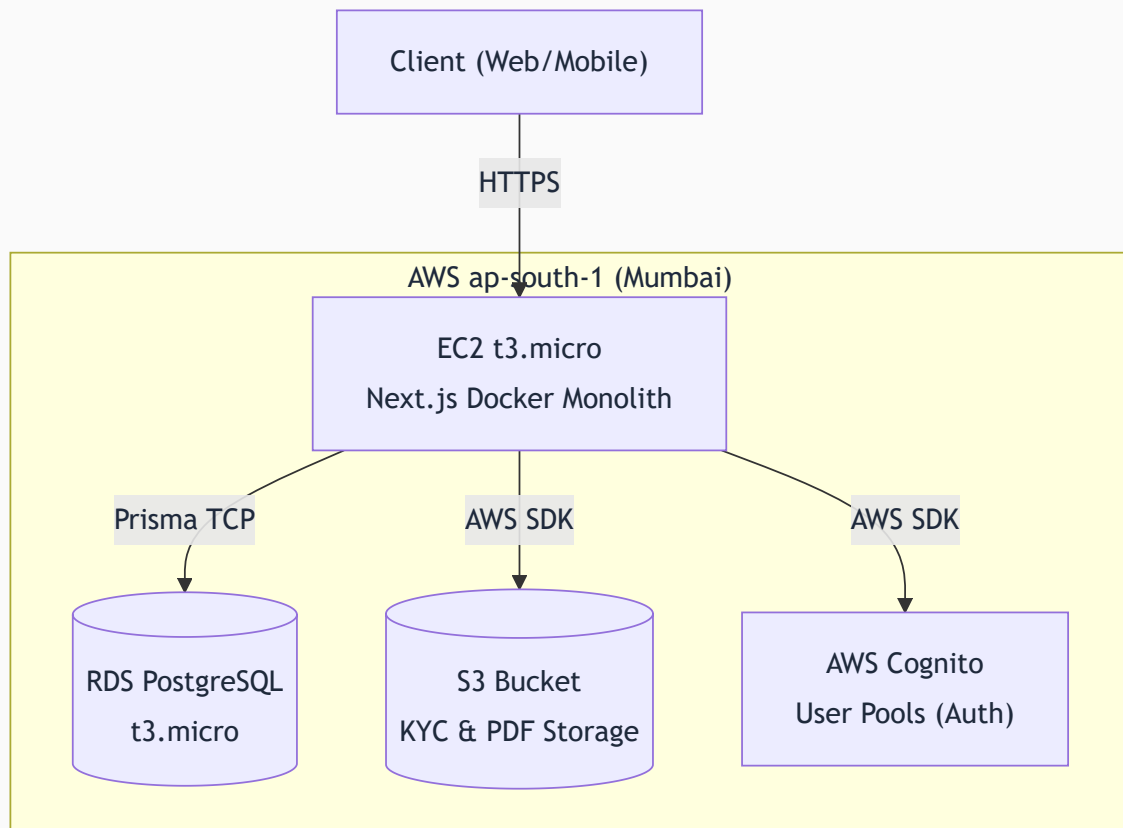
1. Migration Architecture Overview

1.1 What Stays & What Changes

Because Staye is built on **Next.js (App Router)** and utilizes the **Prisma ORM**, the core business logic remains entirely intact.

- **What DOES NOT change:** Database schemas, Prisma queries (`prisma.tenant.findUnique()`), API route business logic, and UI components.
- **What MUST be rewritten:**
 1. Authentication logic (migrating from `supabase.auth` to AWS Cognito).
 2. File upload logic (migrating from `supabase.storage` to `@aws-sdk/client-s3`).
 3. Environment variables and CI/CD pipelines.

1.2 Target Topology (Phase 1 MVP)



2. Sprint 1: Cloud Foundation & Infrastructure Provisioning

Objective: Securely configure the AWS environment and provision empty resources. No code changes happen in this sprint.

Step 2.1: Root Security & IAM Configuration

- 1. Secure the Root Account:** The founders have created the AWS Root account. Immediately enable MFA (Multi-Factor Authentication) on the Root account. Do not generate Access Keys for the Root account.
- 2. Create IAM Policies:**
 - Navigate to IAM → Policies.
 - Create a developer policy that grants access to EC2, RDS, S3, Cognito, and ECR, but explicitly denies billing access and IAM user creation.
- 3. Create IAM Users:**
 - Create an IAM User named `stacy-developer`.
 - Attach the developer policy.
 - Generate an AWS Access Key and Secret Key for this user (store securely in a local `.env` file). **Never push these to GitHub.**

Step 2.2: VPC & Network Security

- 1. VPC Setup:** Utilize the default AWS VPC in `ap-south-1` (Mumbai).
- 2. Security Groups (Firewalls):**
 - `sg-web` (**Attached to EC2**): Allow Inbound HTTP (80) and HTTPS (443) from anywhere (`0.0.0.0/0`). Allow SSH (22) ONLY from the developer's static IP.
 - `sg-db` (**Attached to RDS**): Allow Inbound PostgreSQL (5432) ONLY from the `sg-web` security group. **Do not expose RDS to the public internet.**

Step 2.3: Resource Provisioning

- 1. Amazon RDS (Database):**
 - Engine: PostgreSQL 15 or higher.
 - Instance Class: `db.t3.micro` (Free tier eligible) or `db.t4g.micro`.
 - Storage: 20GB gp2.
 - Public Access: Set to **No**.

2. Amazon S3 (Storage):

- Create bucket: `staye-production-documents`.
- Block all public access (documents will be served via pre-signed URLs from the Next.js API, not directly from the bucket).

3. Amazon Cognito (Auth):

- Create a new User Pool.
- Sign-in options: Email and Password.
- Uncheck "Allow users to sign themselves up" (Tenant accounts are created via the booking flow, not a public signup page).

4. Amazon ECR (Container Registry):

- Create a private repository named `staye-backend`.
-

3. Sprint 2: Codebase Refactoring (Moving off Supabase)

Objective: Rip out Supabase SDK dependencies and wire the Next.js application to the newly provisioned AWS resources.

Step 3.1: Database Connection Update

1. Obtain the Endpoint URL from the newly created AWS RDS instance.
2. Update the `.env` file:

```
# Old: DATABASE_URL="postgresql://postgres:password@db.supabase.co:5432/postgres"
# New:
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@your-rds-endpoint.ap-south-1.rds.amazonaws.com:5432/staye_db"
```

3. Run `npm run prisma db push` (or `migrate deploy`) against the new AWS RDS database to generate the tables.

Step 3.2: Authentication Migration (Cognito)

1. **Remove Supabase Auth:** Uninstall `@supabase/supabase-js` and `@supabase/ssr`.
2. **Install AWS Auth:** Install `aws-amplify` (preferred for Cognito frontend logic) or configure `next-auth` (Auth.js) with the Cognito provider.
3. **Refactor Middleware:**
 - Open `middleware.ts`.
 - Replace the Supabase session check with a Cognito JWT verification logic. If no valid Cognito token is found, redirect to `/login`.
4. **Refactor Login Page:** Update the UI to pass email/password to Cognito instead of Supabase.

Step 3.3: Storage Migration (S3)

1. **Install SDK:** `npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner`
2. **Refactor Upload Logic:**
 - Locate all API routes handling KYC document and Stay Pass uploads.
 - Replace Supabase `upload()` with AWS SDK `PutObjectCommand`.
3. **Refactor Fetch/View Logic:**

- Locate where PDFs and images are rendered.
 - Replace Supabase public URL logic with AWS SDK `getSignedUrl(new GetObjectCommand( ... ))` with a 1-hour expiration.
-

4. Sprint 3: Dockerization & CI/CD Pipeline

Objective: Containerize the application and automate the deployment to EC2 so developers never have to manually transfer files.

Step 4.1: The Dockerfile

Create a production-ready `Dockerfile` in the root of the project utilizing multi-stage builds to keep the image size minimal.

```
# Base node image
FROM node:18-alpine AS base
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Build stage
FROM base AS builder
COPY . .
# Generate prisma client for linux
RUN npx prisma generate
RUN npm run build

# Production stage
FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/package.json ./
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public

EXPOSE 3000
CMD ["node", "server.js"]
```

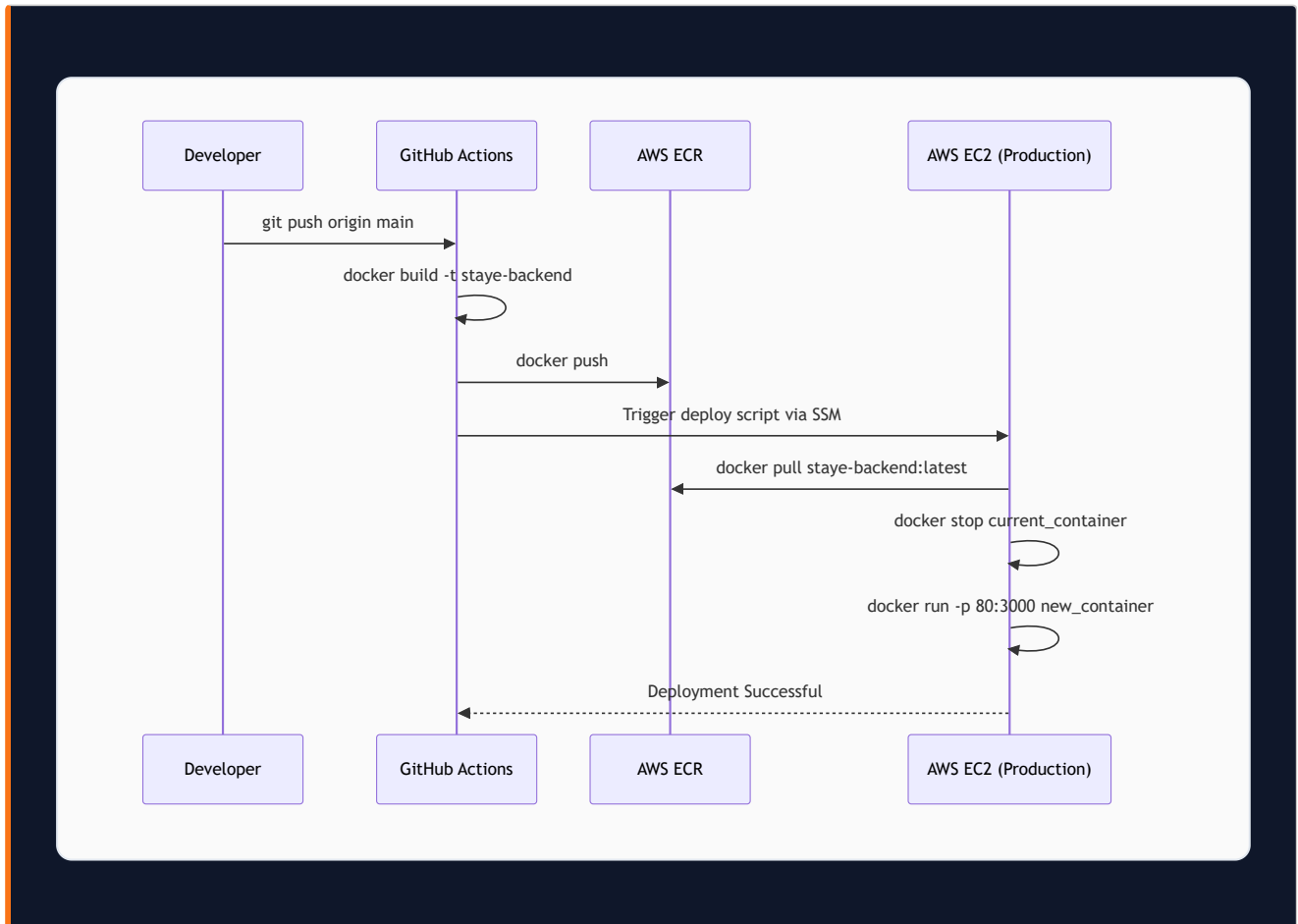
Note: Ensure `output: 'standalone'` is set in `next.config.js`.

Step 4.2: GitHub Actions CI/CD

Create `.github/workflows/deploy.yml`. The pipeline must execute the following steps automatically when code is pushed to the `main` branch:

1. **Build:** Run `docker build`.

2. **Authenticate:** Use `aws-actions/configure-aws-credentials` to log into AWS.
3. **Push:** Push the built Docker image to the AWS ECR repository.
4. **Deploy:** Use AWS Systems Manager (SSM) Run Command to trigger a script on the EC2 instance that:
 - Pulls the latest image from ECR.
 - Stops the old container.
 - Starts the new container injecting secrets from SSM Parameter Store.



5. Sprint 4: Data Cutover & Go-Live

Objective: Safely migrate existing production data from Supabase to AWS with minimal downtime, and flip the DNS switch to go live.

Step 5.1: The Maintenance Freeze

1. Schedule a 2-hour maintenance window during off-peak hours (e.g., 2:00 AM IST).
2. Deploy a maintenance banner to the frontend to prevent users from modifying data.
3. Revoke application access to the Supabase database to guarantee no new writes occur during the copy process.

Step 5.2: Database Migration

1. **Export from Supabase:** Use `pg_dump` to extract the entire database schema and data.

```
pg_dump -U postgres -h db.supabase.co -F c -d postgres > staye_backup.dump
```

2. **Import to AWS RDS:** Use `pg_restore` to push the data into the new RDS instance.

```
pg_restore -U postgres -h your-rds-endpoint.amazonaws.com -d staye_db < staye_backup.dump
```

Step 5.3: Storage Migration

1. Write a temporary Node.js script using the Supabase SDK to list and download all existing KYC documents and PDFs to a local machine.
2. Extend the script to upload those downloaded files directly into the new AWS S3 `staye-production-documents` bucket, maintaining the exact same folder structure and filenames.

Step 5.4: Auth User Cutover

Warning: Passwords cannot be exported from Supabase due to hashing security.

1. Export the list of user emails and roles from Supabase Auth.
2. Write a script to create these users in AWS Cognito using `adminCreateUser`.
3. Set their accounts to require a password reset on first login.

- 4. Tenant Communication:** Send an automated email/SMS to all existing users: *"Staye has upgraded our systems. Please click here to set a new password for your account."*

Step 5.5: DNS Flip (Go-Live)

1. Log into your domain registrar (or Route 53 if managing DNS there).
 2. Update the `A` record (or `CNAME`) for `app.staye.com` to point to the Elastic IP of the AWS EC2 instance.
 3. Once DNS propagates, verify the platform is running on the AWS infrastructure.
 4. Disable the old Supabase project to prevent accidental usage and stop billing.
-

End of Runbook